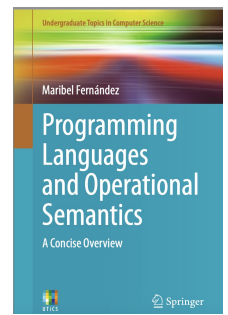


Chapter 1

Introduction



This book is concerned with programming languages. Programming languages are tools for writing software; they enable the user to communicate with the computer.

There are hundreds of programming languages in use in the world today. Many of them have evolved from older languages, and are likely to continue evolving in the future. In order to understand this plethora of languages and adapt easily to changes in the languages we are familiar with, it is useful to study the concepts underlying programming languages in general. We will study the main components of programming languages, and some of the tools that are used to describe in a precise way the behaviour of their main constructs. Our goal will be not only to understand how programming language primitives work but also to be able to describe their behaviour in a detailed and precise way. This is important in order to:

- understand the differences between apparently similar constructs in different languages;
- be able to choose a suitable programming language for each application;
- increase our ability to learn new languages and our ability to adapt to changes in existing languages;
- design new languages (not just programming languages: the general concepts underlying programming language design can also be applied to the design of other kind of languages, for instance, software modelling languages, languages to specify security policies, or user-interfaces for software applications).

In this book, we focus on general purpose languages as opposed to domain specific languages. General purpose languages are programming languages that can be used to program a variety of applications, whereas domain specific languages are special purpose languages designed to be used in specific domains. Special purpose languages include specific constructs to facilitate programming particular kinds of applications. For example, one of the first domain specific languages was GPSS (General Purpose Simulation System [1]), designed in the 1960s for systems simulation. More modern examples of domain specific languages include eXtensible Stylesheet Language Transformations (XSLT [2]), a language defined by the World

Wide Web Consortium in the 1990s to transform XML documents, and JavaServer Pages Standard Tag Library (JSTL [3]), a markup language with programming capabilities. Domain specific languages usually have a narrow application domain, and have their own specific programming techniques and design methodologies, which cannot be easily compared with those of general purpose languages. Since our aim is to analyse the features of various programming languages in order to be able to understand their differences and make informed choices, in the rest of the book we will only consider general purpose languages.

1.1 Programming Languages and Software Engineering

Programming languages are used in several phases in the software development process: in the implementation phase obviously, but also in the design phase to describe the decomposition of the system into modules that later will be coded and tested.

There are various design methods in use, for example: top-down design, functional design, object-oriented design. Some languages provide better support for design methods than others:

- Older languages such as Fortran (the first version was reported in 1954 [4]) do not support any specific design method.
- More modern languages were designed to support a specific design method: for example, Pascal [5] was designed to support top-down development and structured programming, Haskell [6] supports functional design, Java [7] supports object-oriented design and programming.

If the design method is not compatible with the programming language in which the application will be coded, the programming effort increases. For large applications in particular, it is advisable to choose a design method and a programming language that are compatible. The style of the programming language should be reflected in the design of the software application. In this way, the design abstractions can easily be mapped into program components. For instance, we should think in terms of functions if we are going to use a functional language, or in terms of objects if we are going to use an object-oriented language.

1.2 Programming Paradigms

A programming language induces a particular style of programming and a particular set of methods and techniques to reason about programs, called a *programming paradigm*. Programming languages are usually classified according to the paradigm they support. The main classes of programming languages are:

- **Imperative Languages:** Programs are decomposed into *computation steps* (also called commands, statements, or instructions), reflecting the step-wise execution of programs in usual hardware. Subprograms (also called routines, procedures, etc.) are used to build programs in a modular way. Programs written in imperative languages give accurate descriptions of *how* to solve a given problem. For example: Fortran, Algol [8], Pascal, C [9], Python [10] and Java are imperative languages.
- **Functional Languages:** Programs are functions, which can be composed to build new functions as in the mathematical theory of functions (the basis of these languages). This class of languages is also called declarative, since the focus is on *what* should be computed, not *how* it should be computed. Functional languages emphasise the use of expressions, which are evaluated by simplification. For example: Haskell, SML [11], Caml [12] and Clean [13] are functional languages. LISP [14], introduced by John McCarthy in the 1950s, is considered to be the ancestor of all functional programming languages.
- **Object-oriented Languages:** Programs are collections of objects which can only be accessed through the operations (or methods) provided for them, and are usually hierarchically organised. For example: Java and Python are object-oriented languages. An object can be thought of as an entity combining data (fields) and operations (methods). In the object-oriented design methodology, the designer produces a hierarchical description of the structure of the system, which is the basis for the implementation in an object-oriented language. Object-orientation is sometimes considered as a feature of imperative languages, however, it can also be found in functional languages and it can be combined with logic languages too. Therefore we see it as a paradigm of programming in its own right. However, this class of languages is out of the scope of this book.
- **Logic Languages:** Programs describe a problem rather than defining an algorithmic implementation. Again this class of languages is declarative, since the focus is on the specification of the problem that needs to be solved and not on how it is solved. A logic program is a description (usually using the language of first-order logic) of facts and properties about a problem. The most well-known logic programming language is Prolog [15], which was designed by Colmerauer, Roussel and Kowalski in the 1970s. More modern logic programming languages combine logic programming and constraint-solving.

This list is not exhaustive. There are other classes of languages, for example, real-time programming languages, interaction-based languages, and scripting languages, such as JavaScript [16] and PHP [17] used in web programming. In this book we will focus on imperative, functional and logic languages.

1.3 Components of a Programming Language

A programming language has three main components:

1. **Syntax:** defines the form of programs. It describes the way expressions, commands and declarations are built and put together to form a program.

2. *Semantics*: gives the *meaning* of programs. It describes the way programs behave when they are executed.
3. *Implementation*: a software system that can read a program and execute it in a computer. It usually consists of a compiler or interpreter, plus a set of tools, such as editors and debuggers, which help programmers write and test programs. Programming languages usually include a runtime support, which provides functionalities such as memory management primitives (e.g., a garbage collector), mechanisms to implement calls to subprograms including parameter passing, etc.

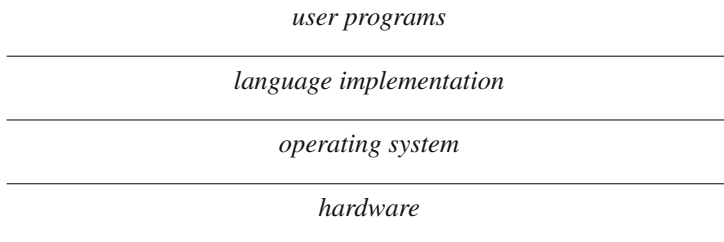
We will describe each of these components in this section, starting from implementation. But first, to help understanding how programming languages are implemented, we give a brief overview of computer architecture.

1.3.1 Computer Architecture

The main components of a *Von Neumann machine* (the most popular kind of computer) are:

- Memory, which contains data and machine instructions represented as sequences of bits;
- Processor (also called Central Processing Unit, CPU), which consists of an arithmetic logic unit with a set of processor registers, and a control unit with an instruction register and program counter;
- Peripherals, which may include a keyboard, display, mouse, printer, etc;
- File systems, which permit to store data externally.

The set of machine instructions for arithmetic and logic operations used by the processor is usually called *machine language*. The *operating system* supplies higher-level primitives than those of the machine language (e.g. input/output operations, file management system, editors). For example, Linux, Unix, MS DOS and Mac OS are operating systems. Language implementations are built on top of these, and user programs or applications form the top layer, as shown in the following diagram.



The three layers hardware, operating system and language implementation are sometimes called a *virtual machine*. This is because if we have an implementation of say, the language C, then by abstracting the intermediate levels we can think of

the system as a machine that executes C instructions. In other words, we can see it as a computer whose machine language is C.

1.3.2 Language Implementation

Programming languages can be classified as low level or high level, depending on how close they are to machine language. Low-level languages are machine dependent. For example, machine instructions, and even assembly languages in which names are given to machine instructions and data, are low-level languages. In this book we are concerned with high-level programming languages, which are (more or less) independent from the machine.

Machine independence is good in terms of portability (the ease with which code can be moved from one machine to another), but bad in that the hardware of the computer is not able to execute the language commands directly. Therefore, if we use high-level languages we need special programs (which are themselves executable) to translate our high-level programs into executable code, or simply to read and execute our high-level programs. In other words, we need an implementation of the language.

High-level programming languages can be implemented by:

- *Compiling* programs into machine language: a compiler is a program that can read a high-level program and produce executable code for it;
- *Interpreting* programs: an interpreter is a program that can read a high-level program and execute its instructions;
- using a *Hybrid Method*, which combines compilation and interpretation, that is, the program is compiled into intermediate code, which it is later interpreted.

Although compilation, interpretation and the hybrid method are fundamentally different, they share some features. For instance, the three methods require first reading the program and identifying its components. We will describe the compilation method below, and then highlight the distinguishing features of the other two methods in relation to compilation.

1.3.2.1 Compilation

A compiler is a program that translates a *source language* (usually a high-level language) into *object language* (usually machine language). The output of the compiler is a program that can be executed directly on the computer, although often other programs are needed (for example, a *linker*) in order to include code from libraries into an executable binary program.

Once our original program has been compiled, we can run it by executing the binary program produced by the compiler (and possibly the linker). We can of course run the program many times without having to repeat the compilation process.

Usually compilers include optimisation phases, where the machine code generated is optimised to improve its efficiency, both in terms of memory use and execution time (although there is often a trade-off between the two). As a result, this method of implementation of programming languages provides fast program execution.

The main phases of the compilation process are:

1. *Lexical Analysis.*

The building blocks of a programming language are lexical units called *tokens*. For example, identifiers (such as names of variables), keywords (such as *begin*, *if*, *then*, *else*, etc), separators (such as comma, colon, semicolon) are tokens. The lexical analyser, which is sometimes called the *lexer*, reads a file containing a program (that is, reads the sequence of characters that forms a program) and identifies a sequence of tokens, which are then passed to the syntax analyser. The lexical analyser may also produce an error message, if the characters in the input do not correspond to any of the tokens of the language.

2. *Syntax Analysis (Parsing).*

The syntax analyser, which is often called the *parser*, reads a sequence of tokens and produces a parse tree (also called syntax tree) for the program, or an error message if the sequence of tokens does not obey the rules of the language. In order to check that the sequence of tokens corresponds to a syntactically correct program, the parser uses a formal definition of the syntax of the language, usually given by a *grammar*.

3. *Type and Semantics Analysis.*

Some programming languages are *typed* and at compile time programs are checked to detect type errors. Programs that do not respect the typing rules of the language are rejected by the compiler.¹ Usually the syntax tree generated by the parser is used for type checking, and also for (optional) code optimisations.

4. *Machine Code Generation.*

The code generator produces machine code, more precisely, it generates a machine-language program equivalent to the optimised syntax tree (and to the original source program).

At this point, we are ready to execute the program. To actually run the program, we need to execute the object code produced by the compiler (and possibly by the linker). Machine code is executed in the CPU, but resides in memory. Therefore, to be executed, each instruction needs to be brought to the CPU, decoded and executed. This process is called the *fetch-execute cycle*. During the fetch-execute cycle,

¹ In some typed languages the compiler does not check types but introduces code so that types can be checked at run time.

instructions are moved one by one from the memory, where the program is stored, to the processor. The fetch-execute cycle consists of the following steps:

- Initialise the Program Counter (also called Instruction Pointer), in a register of the machine;
- Repeat:
 - fetch the instruction to which the Program Counter points;
 - increment the Program Counter;
 - decode and execute the instruction.

When the execution of the program finishes, control returns to the operating system (of course, in a system where more than one program is running the process is more complex).

1.3.2.2 Interpretation

In this case there is no translation to machine code: an interpreter is a program that reads, analyses and directly executes the instructions of a high-level language (or produces an error message when a lexical, syntactic or semantic error is detected).

The interpreter simulates a machine whose fetch-execute cycle deals with high-level program instructions; in other words, the interpreter provides us with a *virtual machine* whose machine code is a high-level language.

Interpretation has some advantages over compilation: interpreters are easier to implement than compilers and provide better error messages, so interpreted programs are easier to debug. However, there are also some disadvantages: there is limited scope for optimisation if instructions have to be directly executed; for this reason in general the execution of programs in interpreted languages is slower.

Simpler languages, and prototyping languages, are usually interpreted (for example: LISP and Unix Shell Scripts are interpreted).

Some languages have both an interpreted and a compiled version. The interpreter can be used for didactic purposes, to write prototype software applications, or to develop and debug programs that are later compiled to increase execution speed. For example, for the study of functional programming languages later in this book we will use an interpreter for Haskell (called Hugs) but compilers are also available (for example, ghc).

1.3.2.3 Hybrid Implementation

Some languages are implemented by translation (compilation) to an intermediate simpler language, instead of machine language. This intermediate language is then interpreted.

The advantage of this solution is portability: in this way programs can be executed in any machine that has an interpreter for the intermediate language.

There are many examples of this technique. Pascal was one of the first languages implemented in this way. Many functional languages are translated to

abstract machine code, which is interpreted. Java compilers also produce code in an intermediate language (bytecode) and programs can then be executed in any machine with a bytecode interpreter. Just-in-time implementations are hybrid, but instead of simply interpreting the intermediate code, a Just-in-time implementation compiles subprograms in the intermediate language the first time they are called; the code will then be reused in subsequent calls.

We will not study language implementation techniques in more detail in this book (the interested reader can find in the Further Reading section at the end of this chapter references to books where these topics are studied in detail). The rest of this chapter will introduce the tools used to describe the syntax and semantics of programming languages.

1.3.3 Syntax

The *syntax* of a programming language is concerned with the *form* of programs. It specifies the set of characters that can be used to write programs, and provides rules indicating how to form lexical units (such as keywords, identifiers, constants, etc.) and how to combine them to build expressions, commands, etc.

We have to distinguish between *concrete syntax* and *abstract syntax* of programming languages. The concrete syntax describes which sequences of characters are well-formed programs, whereas the abstract syntax describes the syntax trees that represent programs. In other words, the concrete syntax describes the actual programs, whereas the abstract syntax describes their representation (the output of the syntax analyser). For this reason, definitions of concrete syntax often include explanations about things like the use of indentation and how comments should be written. The abstract syntax does not deal with these details, since indentation and comments are not included in the representation of the program. Below we give more examples to illustrate the differences between concrete and abstract syntax, using arithmetic expressions. First we need to introduce some terminology.

Formally, a language is a set of *words* built out of characters from a given alphabet. In the case of programming languages, the words are programs. In order to describe the set of words that belong to the language, we use *grammars*.

The grammar of a programming language describes the set of programs that are syntactically correct; it specifies the concrete syntax of the language.

Grammars can be used as generators, that is, as a mechanism to generate the words of a language, or as checkers, that is, as a tool to check that a given word belongs to a given language.

A grammar is specified by:

- an alphabet: a set of symbols containing all the characters, or *terminals*, of the language, plus additional *non-terminal* symbols used to describe the generation of words;

- a set of rules of the form $A \rightarrow s$, also called *productions*, where A is a non-terminal and s a sequence of terminals and non-terminals, which indicate how words are built; and
- a distinguished non-terminal (called *initial symbol*), which is used as a starting point for the generation of words.

Sometimes the alphabet is not explicitly given, since it can be deduced from the rules of the grammar. Also, the initial symbol is usually omitted: it is assumed to be the left-hand side of the first rule. Rules with the same left-hand side are often grouped: we write $A \rightarrow s_1 \mid s_2$ as an abbreviation for the two rules $A \rightarrow s_1$ and $A \rightarrow s_2$.

Example 1.1 (Arithmetic Expressions)

The following grammar defines the concrete syntax of a simple language of arithmetic expressions, where the alphabet contains the terminals 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, −, *, *div*, and the non-terminals *Exp*, *Num*, *Op*, *Digit*. *Exp* is the initial symbol. The rules to form expressions are:

$$\begin{aligned}
 \textit{Exp} &\rightarrow \textit{Num} \mid \textit{Exp Op Exp} \\
 \textit{Op} &\rightarrow + \mid - \mid * \mid \textit{div} \\
 \textit{Num} &\rightarrow \textit{Digit} \mid \textit{Digit Num} \\
 \textit{Digit} &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
 \end{aligned}$$

To generate the words in the language, we use the grammar rules from left to right. In this example we have grouped the rules with the same left-hand side. The symbol $\mid$ in the right hand side of a rule is understood as an “or”. For instance, the first rule says that an expression is generated by the non-terminal *Num*, or has the form *Exp Op Exp*. So, this grammar indicates that there are only two kinds of expressions in the language: an expression is a number, or is formed by an expression followed by an operator, followed by another expression.

More precisely, to build an expression, we start with the initial symbol, in this case the non-terminal *Exp*, and use the rules from left to right until we obtain a word containing only terminals.

For instance, we can generate the expression $2 * 3 + 4$ as follows:

$$\begin{aligned}
 \textit{Exp} &\rightarrow \textit{Exp Op Exp} && \rightarrow \textit{Num Op Exp} \\
 &\rightarrow \textit{Digit Op Exp} && \rightarrow 2 \textit{ Op Exp} \\
 &\rightarrow 2 * \textit{Exp} && \rightarrow 2 * \textit{Exp Op Exp} \\
 &\rightarrow 2 * \textit{Num Op Exp} && \rightarrow 2 * \textit{Digit Op Exp} \\
 &\rightarrow 2 * 3 \textit{ Op Exp} && \rightarrow 2 * 3 + \textit{Exp} \\
 &\rightarrow 2 * 3 + \textit{Num} && \rightarrow 2 * 3 + \textit{Digit} \\
 &\rightarrow 2 * 3 + 4
 \end{aligned}$$

A sequence of applications of rules that starts with the initial symbol and ends with a sequence of terminals (that is, a word in the language) is called a *derivation*

of the word. For example, the sequence of steps from Exp to $2 * 3 + 4$ shown above is a derivation.

The problem with the definition of arithmetic expressions given in Example 1.1 is that there are several ways of deriving an expression: the grammar is *ambiguous*. Ambiguity is problematic, because the parser for the language, which translates the concrete syntax into abstract syntax, produces different outputs depending on the derivation used to generate the expression. For example, the expression $2 * 3 + 4$ can also be derived as:

$$\begin{array}{ll}
 Exp \rightarrow Exp \ Op \ Exp & \rightarrow Exp \ Op \ Num \\
 \rightarrow Exp \ Op \ Digit & \rightarrow Exp \ Op \ 4 \\
 \rightarrow Exp \ + \ 4 & \rightarrow Exp \ Op \ Exp \ + \ 4 \\
 \rightarrow Exp \ Op \ Num \ + \ 4 & \rightarrow Exp \ Op \ Digit \ + \ 4 \\
 \rightarrow Exp \ Op \ 3 \ + \ 4 & \rightarrow Exp \ * \ 3 \ + \ 4 \\
 \rightarrow Num \ * \ 3 \ + \ 4 & \rightarrow Digit \ * \ 3 \ + \ 4 \\
 \rightarrow 2 \ * \ 3 \ + \ 4 &
 \end{array}$$

The first derivation, shown in Example 1.1, identifies the operator $*$ as the main operator in the expression, with arguments 2 and $3 + 4$. It produces a syntax tree corresponding to $2 * (3 + 4)$, that is, the root of the generated syntax tree represents the operator $*$. The second derivation selects $+$ as the main operator, with arguments $(2 * 3)$ and 4, so it produces a syntax tree corresponding to $(2 * 3) + 4$, where the root of the syntax tree represents the operator $+$.

Since compilers generate code for an expression based on its syntax tree, this means that depending on the derivation chosen by the compiler we could get different code and possibly different results for the same expression (as in this example). This is of course something that must be avoided; for this reason, various techniques have been devised to ensure that the grammars defining the syntax of programming languages are non-ambiguous (unfortunately, there is no algorithm to transform an arbitrary grammar into an equivalent non-ambiguous one; the problem is undecidable in general).

The grammar given in Example 1.1 could be transformed into a non-ambiguous one by taking into account the priority of the operators and introducing brackets in the concrete syntax of the language. For example, a standard non-ambiguous grammar for arithmetic expressions defines separate syntactical categories for terms and factors:

$$\begin{array}{ll}
 Exp & \rightarrow Term \mid Term \ LowOp \ Exp \\
 LowOp & \rightarrow + \mid - \\
 Term & \rightarrow Factor \mid Factor \ HighOp \ Term \\
 HighOp & \rightarrow * \mid div \\
 Factor & \rightarrow (\ Exp \) \mid Num \\
 Num & \rightarrow Digit \mid Digit \ Num \\
 Digit & \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
 \end{array}$$

The translation from concrete syntax to abstract syntax is performed by the lexer and parser during the first phase of the compilation or interpretation process.

To specify the form of the abstract syntax trees generated by the parser, we can also use a grammar. However, the alphabet of this grammar will be different from the alphabet of the grammar specifying the concrete syntax: now the terminals are the tokens recognised by the lexer (the input for the parser). Moreover, since this grammar specifies trees (representing programs) rather than strings, we need a notation for trees. We will introduce it with an example.

Example 1.2

The abstract syntax of the language of arithmetic expressions defined in Example 1.1 is defined by the following grammar. The terminal n represents numerical tokens; E and Op are non-terminals. The terminals $+$, $-$, $*$, div are tokens corresponding to the arithmetic operators (we have chosen to use the same names for the operators in the concrete and in the abstract syntax).

$$\begin{aligned} E &\rightarrow n \mid Op(E, E) \\ Op &\rightarrow + \mid - \mid * \mid div \end{aligned}$$

The first rule specifies that arithmetic expressions are either a number token or a tree where the root is labelled by a binary operator and the two subtrees are arithmetic expressions (the second rule, defining Op , specifies the kind of operators that can be used). Graphically, we can visualise the form of the tree as:

$$\begin{array}{c} Op \\ / \quad \backslash \\ E \quad E \end{array}$$

This grammar generates all the *abstract syntax trees* representing arithmetic expressions in the language. In this case, according to the grammar, in the syntax trees the leaves are numbers and the internal nodes are labelled with operators. Importantly, there is no ambiguity since the grammar specifies trees rather than strings.

For example, the abstract syntax tree $*(2, +(3, 4))$, where we denote the number tokens directly by their values (2, 3, 4 in this case), represents the expression with concrete syntax $2 * (3 + 4)$. Graphically, this tree is represented as follows:

$$\begin{array}{c} * \\ / \quad \backslash \\ 2 \quad + \\ / \quad \backslash \\ 3 \quad 4 \end{array}$$

We see that the operator $*$ at the root is applied to the arguments 2 and $+(3, 4)$, respectively. The latter is the tree representation of the string $3 + 4$. The expression $(2 * 3) + 4$ will instead be represented by the abstract syntax tree $+(*(2, 3), 4)$, shown below.



In the rest of the book we will only work with the abstract syntax of programming languages. This is because we are interested in the *semantics* of the various constructs of programming languages. The concrete syntax is not relevant from a semantics point of view, although of course when writing a program it is important to know the concrete syntax of the language, as programs that do not respect the constraints will be rejected by the compiler or the interpreter.

1.3.4 Semantics

The *semantics* of a language defines the meaning of programs, that is, how they behave when they are executed on a computer.

Since similar constructs are often used in different languages with different syntax and different semantics, it is important to have a precise description of their behaviour. Variations in concrete syntax are in general superficial, but differences in meaning are important since constructs that are apparently similar may produce very different results.

The semantics of a programming language can be defined at two levels: *static semantics* (also called *typing*), and *dynamic semantics*, or just semantics. We will briefly describe both.

1.3.4.1 Static Semantics

Types can be seen as specifications of programs, and therefore they are part of the semantics of the language, but they are a form of static semantics. The goal is to detect (before the actual execution of the program) programs that are syntactically correct but will give errors during execution.

Example 1.3

Consider a language with arithmetic and Boolean expressions where the abstract syntax is given by:

$$\begin{aligned} E &\rightarrow n \mid b \mid Op(E, E) \\ Op &\rightarrow + \mid - \mid * \mid div \mid \text{and} \mid \text{nor} \end{aligned}$$

Here n represents numbers and b Booleans. The expression $+(1, \text{True})$ is syntactically correct (it can be generated by this grammar), but it is not typeable: attempting to add a number and a Boolean should produce a type error.

Most typed languages require type declarations associating types with program constructs. In some functional languages, types can be *inferred*, which frees the programmer from the need of writing type declarations. We will study type checking and type inference systems in Chaps. 5 and 6.

1.3.4.2 Dynamic Semantics

The dynamic semantics of the language describes the behaviour of programs at execution time. It is often given by informal definitions and explanations in natural language (e.g., English) in language manuals. However, natural language explanations are in general imprecise and incomplete. An alternative is to provide a formal semantics. This is in general more difficult but can be very useful, and even essential in some contexts, for several reasons:

1. **Language Implementation:** A formal definition of the semantics of the language facilitates its implementation. This is because the semantics specifies the behaviour of each construct in abstract terms, providing a full description of the execution process that is independent of the actual machine. Compilers and interpreters are ultimately software applications, and as for any software application, it is easier to write it if we have a clear and complete specification of what it should do.
2. **Programming:** Programmers obviously benefit from a clear description of the constructs in the language. Moreover, the techniques used to give a formal semantics of a programming language are usually accompanied by techniques and tools to reason about programs and to prove properties of programs. Formal verification of programs, which aims at providing proofs of correctness (i.e., proving that a program satisfies its specification), is only possible if a formal description of the behaviour of the program is available.
3. **Language Design:** A formal semantics prevents ambiguities in the constructs and helps detect omissions (a notorious example where an omission remained undetected for some time was C: the evaluation order for function parameters was not specified in the first version of the language). A formal semantics guides the design of the language and suggests improvements and new constructs. For example, the use of the λ -calculus as a tool to give a formal semantics to programming languages has influenced the design of functional and object-oriented languages.

However, formal semantic descriptions of general programming languages can be hard to produce and difficult to read, so they are usually available for parts of the language only, or for special purpose languages (for instance, they are a standard feature in hardware description languages).

There are different approaches to the formal definition of the semantics of programming languages. The main semantic styles are:

- *Denotational*: The meaning of expressions, and in general, the meaning of the constructs in the language, is given in an abstract way using a mathematical model for the language. The semantics describes the *effect* of each construct. This approach has some similarities with the approach used in natural sciences to explain natural processes, where a mathematical model of the system under study is built, then natural processes are explained in terms of the model. In the case of programming languages, the model deals with computation, and it should be sufficiently powerful to allow us to explain the meaning of all the constructs in the language.
- *Axiomatic*: The meaning of programs is given by describing the properties that hold before and after the execution of the program, using axioms and deduction rules in a specific logic. For each kind of construct, predicates or assertions are given, describing the constraints on program variables before and after the execution of the statement. These are called *preconditions* and *postconditions*. This approach is particularly useful to programmers, since it shows clearly the effect of each program construct. It has been used as a basis for the development of program verification tools.
- *Operational*: The meaning of each construct is given in terms of computation steps. *Transition systems* are used as a tool to give the operational semantics of programs: the execution of the program is described as a sequence of transitions. Transition systems can be presented as abstract machines, where the transitions represent changes of state in the machine, or using axioms and rules to define a transition relation (this is called structural operational semantics). The operational approach was one of the first used to give a formal definition of the semantics of programming languages.

Each style of semantics has its advantages, they are complementary. Operational semantics is very useful for the implementation of the language and for proving correctness of compiler optimisations, since it describes the computation steps that are required to execute each command. On the other hand, denotational semantics and axiomatic semantics are useful to reason about programs and to prove properties of programs.

In this book we will only study operational semantics of programming languages, using transition systems. In the next chapter we give the necessary mathematical background.

1.4 Further Reading

Readers interested in the history of programming languages can find more information in Sebesta's book [18]. The notion of abstract syntax was first used by McCarthy [19] in connection with the language LISP. For more information on compiler techniques, Aho et al.'s [20] book is an excellent reference. Further information on programming language semantics can be found in [21]; for advanced readers, we recommend Winskel's book [22]. Additional references are provided in the following chapters.

1.5 Exercises

- Using the grammar that specifies the concrete syntax for arithmetic expressions in Example 1.1:
 - Show that $1 - 2 - 3$ and $1 + 2 * 3$ are valid expressions in this language, and list their possible results, assuming that the arithmetic operators have the usual semantics.
 - Are $1 + (2 - 3)$ and -1 valid expressions in this language?
 - Using Haskell evaluate the expressions² $1 - 2 - 3$ and $1 + 2 * 3$. From the results obtained, can you explain how Haskell has parsed these expressions? Identify the main operator (i.e., the root of the syntax tree generated by Haskell) and its arguments, in each of the expressions.
- Modify the grammar for the concrete syntax of arithmetic expressions in Example 1.1 so that it precludes leading zeros in numbers.
- Using the grammar given in Example 1.2 to specify the abstract syntax of arithmetic expressions:
 - Show that both $-(-(1, 2), 3)$ and $-(1, -(2, 3))$ are valid. What are their values (assuming that the arithmetic operators have the usual semantics)?
 - Explain why $-(1, 2, 3)$ is not a valid abstract syntax tree according to this grammar.
- Modify the grammars given in Examples 1.1 and 1.2 to specify the concrete and abstract syntax for arithmetic expressions, in order to include a unary subtraction operator. The idea is to allow users to write expressions such as -1 in the concrete syntax.
Now modify the grammars again to take into account the fact that the language should also allow users to write expressions of the form x^y where x and y are two arithmetic expressions.

² Simply use the interactive evaluator in your browser, see <http://www.tryhaskell.org>. To evaluate an expression, type the expression after the prompt and press the return key.

5. Assume we want to design a language L in which variable identifiers are sequences of letters or numbers, starting with a capital letter. Give a grammar for the concrete syntax of identifiers in L .
6. The following grammar defines the concrete syntax of identifiers in Haskell. Non-terminals are written in *italics*. In the first rule, we use a special notation to represent set difference: $(small\ rest) - keywords$ denotes the strings generated by *small rest* minus the ones generated by *keywords*. In the third rule, the symbol ϵ denotes an empty string.

varid $\rightarrow (small\ rest) - keywords$

small $\rightarrow a \mid b \mid c \mid \dots \mid z$

rest $\rightarrow \epsilon \mid sldq\ rest$

sldq $\rightarrow small \mid large \mid digit \mid ' \mid$

large $\rightarrow A \mid B \mid C \mid \dots \mid Z$

digit $\rightarrow 0 \mid 1 \mid 2 \mid \dots \mid 9$

keywords $\rightarrow case \mid class \mid data \mid default \mid deriving \mid do \mid else \mid if \mid import \mid in \mid infix \mid infixl \mid infixr \mid instance \mid let \mid module \mid newtype \mid of \mid then \mid type \mid where \mid _$

- (a) Explain in English how identifiers are formed.
- (b) We extend the grammar with a rule to specify the form of function declarations in Haskell:

decl $\rightarrow let\ varid\ args = expr1\ in\ expr2$

where *varid* is the name of the function, *args* its arguments, *expr1* its definition and *expr2* the expression where it is used (we omit the grammar rules defining *args*, *expr1*, *expr2*).

For example, the following is a valid function declaration for a function called *double*, with an argument *x*, defined by the expression $2 * x$ and used to compute the double of 3.

let double *x* = $2 * x$ in double 3

The following Haskell programs have syntax errors. Try to run the programs and explain why they produce error messages.³ Using the grammar given, can you identify the errors and suggest ways to correct them?

³ Type each of the lines using the interactive evaluator for Haskell in your browser, see <http://www.tryhaskell.org>, and explain the errors found.

let Double x = 2 * x in Double 3

let dou?ble x = 2 * x in dou?ble 3

let data = 1 in data + 1.

References

1. G. Gordon, The development of the general purpose simulation system (GPSS). Newsletter, in ACM SIGPLAN Notices. Special issue: History of programming languages conference, vol. **13**, pp. 183–198 (1978)
2. World Wide Web Consortium. XSL Transformations (XSLT), version 1.0 (1999), www.w3.org/TR/xslt
3. E. Jendrock, J. Ball, D. Carson, I. Evans, S. Fordin, K. Haase, The Java EE 5 Tutorial. Oracle (first published 2007) (2010)
4. IBM, *Preliminary Report, Specifications for the IBM Mathematical FORMula TRANslating System* (IBM Corporation, New York, 1954)
5. N. Wirth, The programming language Pascal. *Acta Informatica* **1**(1), 35–63 (1971)
6. S.L. Peyton Jones, *Haskell 98 Language and Libraries* (Cambridge University Press, Cambridge, 2003)
7. J. Gosling, B. Joy, G. Steele, *The Java Language Specification* (Addison-Wesley, Boston, 1996)
8. P. Naur, Report on the algorithmic language ALGOL 60. *Commun. ACM* **3**(5) (1960)
9. B.W. Kernighan, D.M. Ritchie, *The C Programming Language*, 2nd edn. Software Series (Prentice Hall, Upper Saddle River, 1988)
10. G. van Rossum, A tour of the Python language, in TOOLS 1997: 23rd International Conference on Technology of Object-Oriented Languages and Systems, 28 July–1 August 1997 (IEEE Computer Society, Santa Barbara, USA, 1998), p. 370
11. R. Milner, M. Tofte, R. Harper, *The Definition of Standard ML* (MIT Press, Cambridge, 1990)
12. P. Weiss, X. Leroy, *Le Langage CAML* (Dunod, Paris, 1999)
13. Clean Development Team. Concurrent Clean Language Report (version 2.2). clean.cs.ru.nl/Documentation.
14. J. McCarthy, P. Abrahams, D. Edwards, T. Hart, M. Levin, *LISP 1.5 Programmer's Manual*, 2nd edn. (MIT Press, Cambridge, 1965)
15. P. Roussel, PROLOG: Manuel de référence et d'utilisation, *Research Report* (Artificial Intelligence Team, University of Aix-Marseille, France, 1975)
16. D. Flanagan, *JavaScript: The Definitive Guide*, 6th edn. (O'Reilly Media, California, 2011)
17. T. Converse, J. Park, *PHP Bible*, 2nd edn. (Wiley, 2002)
18. R. Sebesta, *Concepts of Programming Languages*, 10th edn. (Addison Wesley, Boston, 2012)
19. J. McCarthy, Towards a mathematical science of computation, in IFIP Congress (1962), pp. 21–28
20. A.V. Aho, M.S. Lam, R. Sethi, J.D. Ullman, *Compilers: Principles, Techniques, and Tools* (Prentice Hall, Upper Saddle River, 2006)
21. H.R. Nielson, F. Nielson, *Semantics with Applications: An Appetizer (Undergraduate Topics in Computer Science)* (Springer, Berlin, 2007)
22. G. Winskel, *The Formal Semantics of Programming Languages. Foundations of Computing* (MIT Press, Cambridge, 1993)